

Proof of Concept Technical Solution for the Marconi Law Firm, LLC. (WordPress Website)

Orlando, Florida

Jarred Ward

WardTech

Table of Contents

Inventory.....	6
Custom Network.....	6
IDs and Passwords.....	6
Preface.....	7
Network Topology Diagram.....	8
SSH Install & Access.....	9
OpenSSH Install (Ubuntu).....	9
Node.js Application (Ghost) on Docker.....	9
Update Rocky8-Docker Host Name.....	9
Update Rocky8-Docker.....	9
Install EPEL Packages.....	9
Install Nano Editor.....	9
Docker CE.....	9
Set up stable repository.....	9
Install Docker CE.....	10
Verify Docker version.....	10
Initialize Docker.....	10
Start Docker.....	10
Enable Docker.....	10
Test Docker (hello-world).....	10
Disable SELinux (/etc/selinux.config).....	10
Reboot VM.....	11
Confirm SELinux Status.....	11
Install Ghost Docker Container.....	11
Check for Ghost Container (docker ps).....	11
Ghost Container ID.....	11
NginX Reverse Proxy.....	11
Update Rocky8-Nginx Host Name.....	11
Update Rocky8-Nginx.....	11
Install EPEL Packages.....	11
Install Nano Editor.....	11
Disable SELinux.....	11

Reboot VM.....	12
Confirm SELinux Status	12
Rocky Firewall	12
Stop Firewall	12
Disable Firewall	12
NginX.....	12
Install NginX.....	12
Start NginX.....	13
Enable NginX.....	13
Confirm NginX Status	13
Reverse Proxy for Ghost Site.....	13
Edit NginX configuration file.....	13
Reload NginX service	13
Terminate Docker	13
Delete First Ghost Container	13
Create New Ghost Container	13
Browse to Ghost from Firefox on Ubuntu (10.10.229.10/blog).....	14
WordPress on Ubuntu - LAMP Stack.....	16
Update Ubuntu Host Name.....	16
Update Ubuntu	16
Upgrade Ubuntu	16
Install Nano Editor.....	16
Install Git	16
Install Apache2.....	16
Open Firewall Ports 80 and 443	16
Browse to Apache2 Ubuntu Default Page.....	16
Install MySQL.....	17
Alter root user password (root@localhost)	17
Flush Privileges	17
Exit MySQL.....	17
Install PHP.....	17
Install Required PHP Libraries	17
Install Required MySQL Libraries	17
Enable URL Rewrites (clean URLs).....	17
Restart Apache Service	17
Create a test.php Web Page.....	17
Test the test.php Web Page	18
Database Configuration in MySQL Log into MySQL	18
Create MySQL WordPress Database (WordPressDB)	18
Create MySQL WordPress User (WordPressUser)	18
Grant Privileges to the WordPress Database (WordPressDB) to WordPress User (WordPressUser)	18

Flush Privileges	19
Exit MySQL.....	19
Install WordPress	19
Grant Permission to /var/www/html/ Directory to WordPress User	19
Delete Files from /var/www/html/ Directory	19
Verify /var/www/html/ Directory is Empty	19
Clone WordPress to /var/www/html/ Directory	19
Verify /var/www/html/ Directory Contains WordPress Files	19
Verify Permissions on /var/www/html/ Directory.....	19
Edit Ownership	20
Edit Ownership of and the contents of /var/www/html/ Directory	20
Edit the apache2.conf File.....	20
Override All Default Apache Directives.....	20
Create a .htaccess File in the /var/www/html/.git/ Directory	20
Restart the Apache Service	21
WordPress Configuration.....	21
Configure WordPress.....	21
WordPress Configuration Selections.....	21
Run Installation.....	22
Create an Admin WordPress User.....	22
WordPress Site Selections	23
Test WordPress Website	23
WordPress Security Settings and Configurations.....	25
File Permissions.....	25
Vulnerability	25
Configuration.....	25
Validation	25
Securing wp-config.php	26
Vulnerability	26
Configuration.....	26
Validation	27
Firewall (Shield)	27
Vulnerability	27
Configuration.....	28
Validation	28
Conclusion	29
Appendix A.....	30
NginX Config File	30
Appendix B	30

NginX Access Log File.....30

NginX Error Log File30

Inventory

EQUIPMENT	OPERATING SYSTEM	ADDITIONAL INFO	IP ADDRESS
Router/Custom Network	- (Firewall VM)	-Firewall VM	10.10.229.1
Docker	Rocky 8 (-Docker)	Ghost Container	10.10.229.11
NginX Reverse Proxy	Rocky 8 (-Nginx)	Reverse Proxy	10.10.229.10
WordPress	Ubuntu	LAMP Stack running WordPress	10.10.229.12

Custom Network

NETWORK NAME	SUBNET IP	SUBNET MASK	DNS	GATEWAY
ITE229	10.10.229.0	255.255.255.0	10.10.229.1	10.10.229.1

IDs and Passwords

ACCOUNT	USER ID	PASSWORD
Rocky8-Docker Root User	root	Fullsail1!
Rocky8-Nginx Root User	root	Fullsail1!
Ubuntu Root User	Root	Fullsail1!
MySQL Root User	root@localhost	Fullsail1!
MySQL WordPress User	WordPressUser	Fullsail1!
WordPress Admin	admin	Fullsail1!

Preface

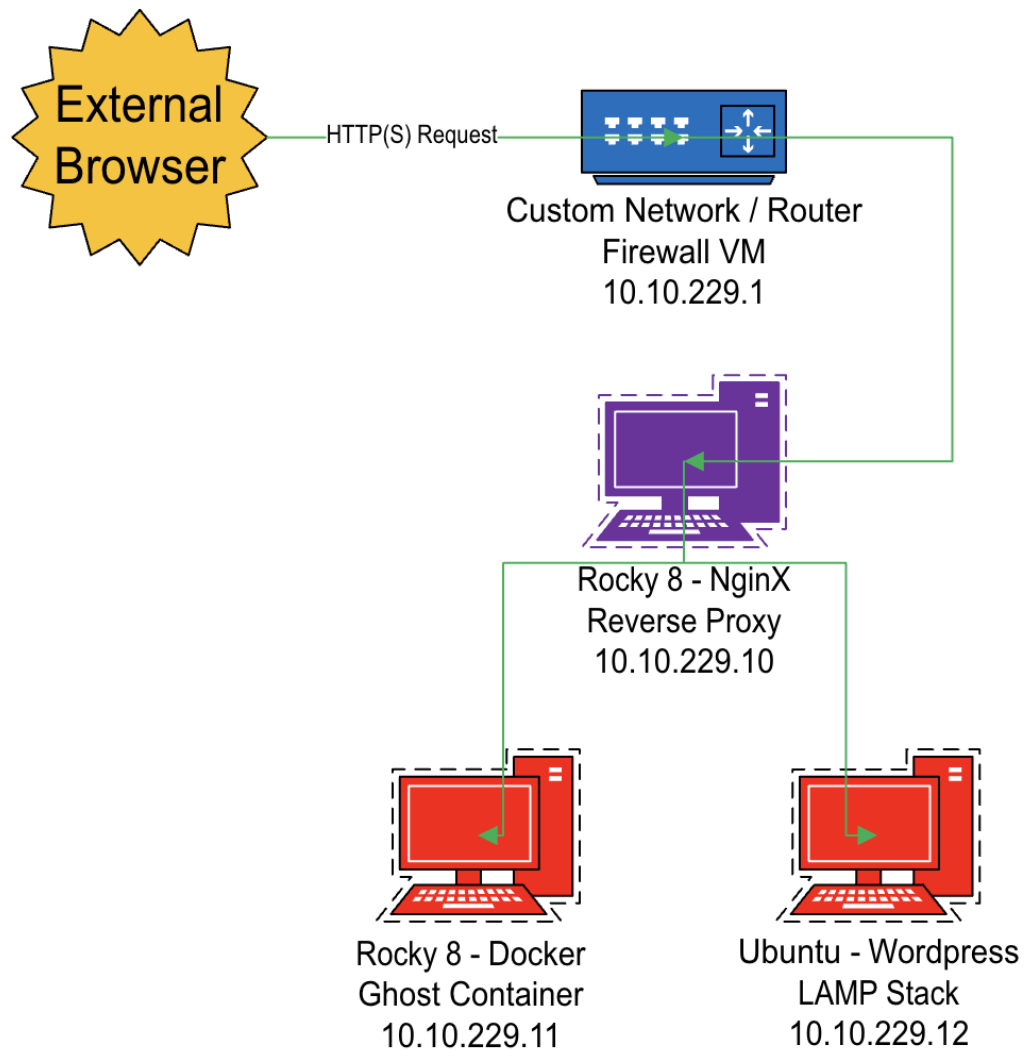
This document will serve as proof of concept to Mr. Marconi for creating his WordPress website for his law firm and as audit documentation.

The purpose of audit documentation is to provide a comprehensive record of the organization's information technology infrastructure and security controls and processes. It plays a crucial role in providing transparency, accountability, and QA/QC regarding an organization's cybersecurity controls and practices. It enables organizations to demonstrate compliance, identify areas for improvement, and make informed decisions to strengthen their overall organizational cybersecurity.

Audit documentation serves several important purposes:

- **Compliance:** Evidence that an organization has undergone a thorough examination of its systems. It helps validate that the organization has implemented appropriate controls to protect its information systems and sensitive data.
- **Validation:** Verification of the effectiveness and adequacy of cybersecurity controls. It provides detailed information about the design, implementation, and operation of these controls, enabling reviewers to assess their reliability and identify any gaps or weaknesses.
- **Records Maintenance:** Historical record of cybersecurity audits conducted over time. It enables organizations to track their progress, identify trends, and evaluate the effectiveness actions taken. It also serves as reference for future audits and allows auditors to understand the current cybersecurity implemented and facilitates a more targeted approach to future cybersecurity updates and audits.
- **Decision-making Support:** Valuable insights and information that can support decision-making processes. It allows management to make informed decisions about allocating resources, prioritizing cybersecurity investments, and addressing identified risks and vulnerabilities.

Network Topology Diagram



SSH Install & Access

OpenSSH Install (Ubuntu)

Install SSH by entering the following command: *sudo apt install openssh-client -y*

```
user@UbuntuWard:~$ sudo apt install openssh-client -y
```

You can SSH into NginX by entering the following command: *ssh root@10.10.229.10*

```
user@UbuntuWard:~$ ssh root@10.10.229.10
```

You can SSH into Docker by entering the following command: *ssh root@10.10.229.11*

```
user@UbuntuWard:~$ ssh root@10.10.229.11
```

Node.js Application (Ghost) on Docker

Update Rocky8-Docker Host Name

To update the host name of Rocky 8 - Docker, enter the following command: *nmtui*

```
[root@DockerWard ~]# nmtui
```

Update Rocky8-Docker

Update Rocky 8 - Docker by entering the following command: *sudo yum update -y*

```
[root@DockerWard ~]# sudo yum update -y
```

Install EPEL Packages

Enter the following command to instal EPEL packages: *sudo yum install epel-releas -y*

```
[root@DockerWard ~]# sudo yum install epel-release -y
```

Install Nano Editor

Install the Nano Editor by issuing the following command: *sudo yum install nano -y*

```
[root@DockerWard ~]# sudo yum install nano -y
```

Docker CE

Set up stable repository

Setting up the repository requires two commands. The first command installs the package manager and the second command adds the repository:

sudo yum install -y yum-utils

```
[root@DockerWard ~]# sudo yum install -y yum-utils
```

sudo yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo

```
[root@DockerWard ~]# sudo yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
```

Install Docker CE

Install Docker CE by entering the following command:

```
sudo yum install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

```
[root@DockerWard ~]# sudo yum install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Verify Docker version

You can verify the Docker version by issuing the following command: *docker -v*

```
[root@DockerWard ~]# docker -v
```

Initialize Docker

Start Docker

Issue the following command to initiate the Docker service: *sudo systemctl start docker*

```
[root@DockerWard ~]# sudo systemctl start docker
```

Enable Docker

Enable Docker with the following command: *sudo systemctl enable docker*

```
[root@DockerWard ~]# sudo systemctl enable docker
```

Test Docker (hello-world)

To test Docker by running the hello-world image, issue the following command: *sudo docker run hello-world*

```
[root@DockerWard ~]# sudo docker run hello-world
```

Disable SELinux (/etc/selinux.config)

To disable SELinux, issue the following command to open the config file with nano text editor:

```
sudo nano /etc/selinux/config
```

```
[root@DockerWard ~]# sudo nano /etc/selinux/config
```

Next, change the status of SELinux to “disabled” as shown below, then save and exit:

```
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled - No SELinux policy is loaded.
SELINUX=disabled
# SELINUXTYPE= can take one of these three values:
#   targeted - Targeted processes are protected,
#   minimum - Modification of targeted policy. Only selected processes are protected.
#   mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

Reboot VM

Reboot the Rocky 8 – Docker VM by issuing the following command: *reboot now*

```
[root@DockerWard ~]# reboot now
```

Confirm SELinux Status

Next, confirm SELinux status by issuing the following command: *sestatus*

```
[root@DockerWard ~]# sestatus
```

Install Ghost Docker Container

Install Ghost Docker Container with this command:

docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.11:3001 ghost

```
[root@DockerWard ~]# docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.11:3001 ghost
```

Check for Ghost Container (docker ps)

Ghost Container ID

Enter the following command to check for the Docker Ghost container: *docker ps -a*

```
[root@DockerWard ~]# docker ps -a
```

NginX Reverse Proxy

Update Rocky8-Nginx Host Name

Update the host name of Rocky 8 – NginX with the following command: *nmtui*

```
[root@NginxWard ~]# nmtui
```

Update Rocky8-Nginx

Next, update the Rocky 8 – NginX VM with the following command: *sudo yum update -y*

```
[root@NginxWard ~]# sudo yum update -y
```

Install EPEL Packages

Install EPEL packages by issuing the following command: *sudo yum install epel-release -y*

```
[root@NginxWard ~]# sudo yum install epel-release -y
```

Install Nano Editor

Install Nano text editor by issuing the following command: *sudo yum install nano*

```
[root@NginxWard ~]# sudo yum install nano
```

Disable SELinux

To disable SELinux, issue the following command to open the config file with nano text editor:

sudo nano /etc/selinux/config

```
[root@NginxWard ~]# sudo nano /etc/selinux/config
```

Next, change the status of SELinux to “disabled” as shown below, then save and exit:

```
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled - No SELinux policy is loaded.
SELINUX=disabled
# SELINUXTYPE= can take one of these three values:
#   targeted - Targeted processes are protected,
#   minimum - Modification of targeted policy. Only selected processes are protected.
#   mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

Reboot VM

Reboot the Rocky 8 – NginX VM by issuing the following command: *reboot now*

```
[root@NginxWard ~]# reboot now
```

Confirm SELinux Status

You can confirm the SELinux status with the following command: *sestatus*

```
[root@NginxWard ~]# sestatus
```

Rocky Firewall

Stop Firewall

Stop the firewall with the following command: *systemctl stop firewalld*

```
[root@NginxWard ~]# systemctl stop firewalld
```

Disable Firewall

Disable the firewall by issuing the following command: *systemctl disable firewalld*

```
[root@NginxWard ~]# systemctl disable firewalld
```

NginX

Install NginX

Install NginX with this command: *sudo yum install nginx -y*

```
[root@NginxWard ~]# sudo yum install nginx -y
```

Start NginX

Enter the following command to start NginX: *systemctl start nginx*

```
[root@NginxWard ~]# systemctl start nginx
```

Enable NginX

Next, enable NginX by entering the following command: *systemctl enable nginx*

```
[root@NginxWard ~]# systemctl enable nginx
```

Confirm NginX Status

You can confirm the NginX status by entering the following command: *service nginx status*

```
[root@NginxWard ~]# service nginx status
```

Reverse Proxy for Ghost Site

Edit NginX configuration file

Edit the NginX configuration file with the following command: *nano /etc/nginx/nginx.conf*

```
[root@NginxWard ~]# nano /etc/nginx/nginx.conf
```

Reload NginX service

Next, reload the NginX service by entering the following command: *systemctl reload nginx*

```
[root@NginxWard ~]# systemctl reload nginx
```

Terminate Docker

Terminate Docker by entering the following command: *docker kill ghost*

You can also use the ID of the ghost container instead, as shown below (your container ID may differ):

```
[root@DockerWard ~]# docker kill 2a07654aabc9
Error response from daemon: cannot kill container: 2a07654aabc9: container 2a07654aabc92fb523ecc54a3d0f022952672517683662fe329478f5d64c7b4e is not running
```

Delete First Ghost Container

Once terminated, delete the ghost container by entering the following command: *docker rm ghost*

You can also use the ID of the ghost container instead, as shown below (your container ID may differ):

```
[root@DockerWard ~]# docker rm 2a07654aabc9
```

Create New Ghost Container

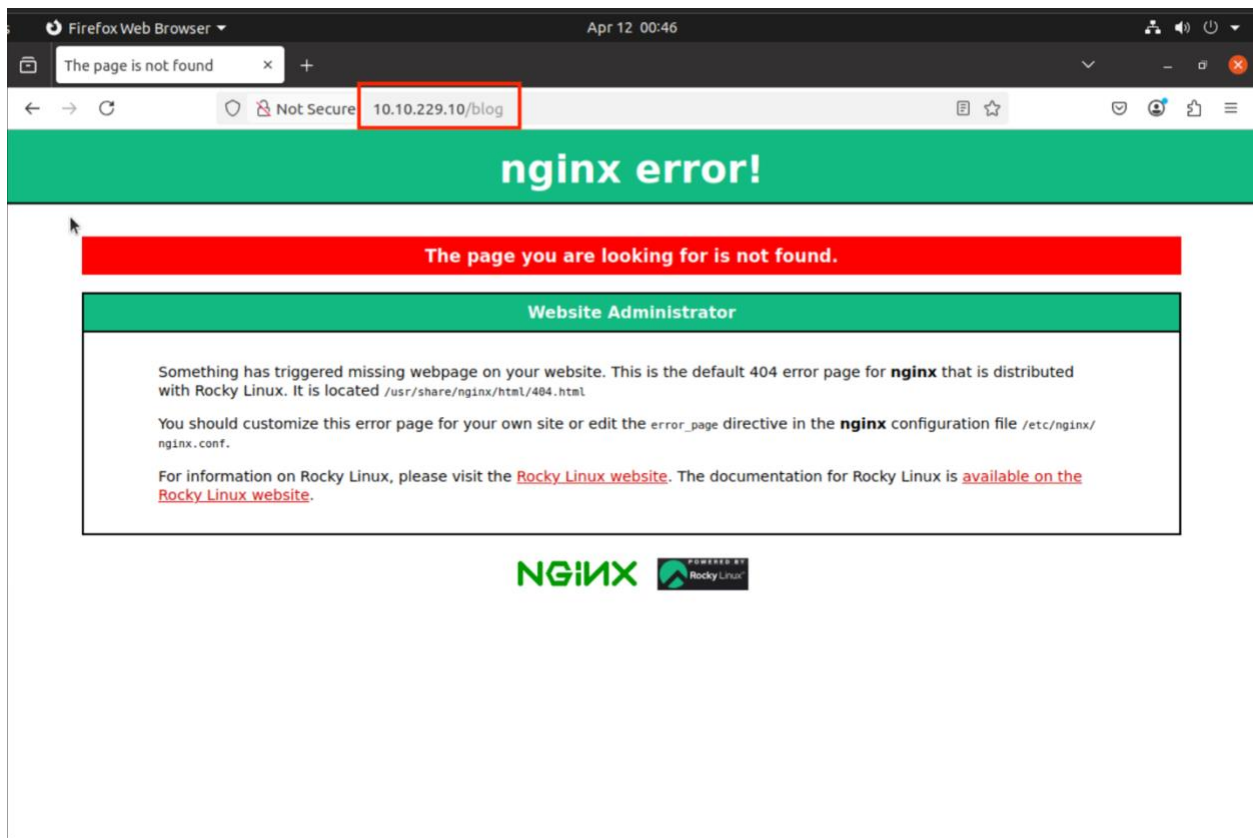
Create a new Ghost container by entering the following command:

```
docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.10/blog ghost
```

```
[root@DockerWard ~]# docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.10/blog ghost
```

Browse to Ghost from Firefox on Ubuntu (10.10.229.10/blog)

With the Firefox browser on the Ubuntu VM, enter the IP address of Rocky 8 – NginX, followed by “/blog”:
10.10.229.10/blog



END OF MILESTONE 1

WordPress on Ubuntu - LAMP Stack

Update Ubuntu Host Name

Update the Ubuntu Host Name by entering the following command: *nmtui*

```
user@UbuntuWard:~$ nmtui
```

Update Ubuntu

Next, update Ubuntu by entering the following command: *sudo apt-get update -y*

```
user@UbuntuWard:~$ sudo apt-get update -y
```

Upgrade Ubuntu

Enter the following command to upgrade Ubuntu: *sudo apt upgrade -y*

```
user@UbuntuWard:~$ sudo apt upgrade -y
```

Install Nano Editor

Install the Nano Text Editor by entering this command: *sudo apt install nano*

```
user@UbuntuWard:~$ sudo apt install nano
```

Install Git

To install Git, enter the following command: *sudo apt install git -y*

```
user@UbuntuWard:~$ sudo apt install git -y
```

Install Apache2

Next, install the Apache Web Server with the following command: *sudo apt install apache2 -y*

```
user@UbuntuWard:~$ sudo apt install apache2 -y
```

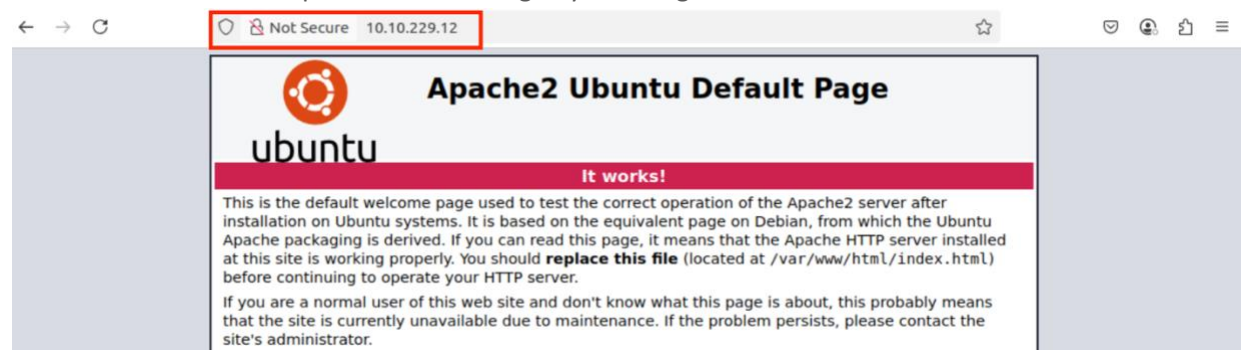
Open Firewall Ports 80 and 443

To open the firewall ports, enter the following command: *sudo ufw allow in "Apache Full"*

```
user@UbuntuWard:~$ sudo ufw allow in "Apache Full"
```

Browse to Apache2 Ubuntu Default Page

You can browse to the Apache2 Default Page by entering this IP into the address bar: *10.10.229.12*



Install MySQL

Install MySQL with the following command: *sudo apt install mysql-server*

```
user@UbuntuWard:~$ sudo apt install mysql-server
```

Alter root user password (root@localhost)

To alter the root user password, enter the following command:

ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!';

```
mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!';
```

Flush Privileges

Enter the following command to flush the privileges and reload the tables: *FLUSH PRIVILEGES;*

```
mysql> FLUSH PRIVILEGES;
```

Exit MySQL

Finally, exit MySQL by entering this command: *exit*

```
mysql> exit
```

Install PHP

Install Required PHP Libraries

To install the required PHP libraries, enter the following command:

sudo apt install php libapache2-mod-php php-mysql

```
user@UbuntuWard:~$ sudo apt install php libapache2-mod-php php-mysql
```

Install Required MySQL Libraries

To install, enter the following command:

sudo apt install php-curl php-gd php-xml php-mbstring php-xmldrpc php-zip php-soap php-intl

```
user@UbuntuWard:~$ sudo apt install php-curl php-gd php-xml php-mbstring php-xmldrpc php-zip php-soap php-intl
```

Enable URL Rewrites (clean URLs)

Enter the following command to enable URL rewrites: *sudo a2enmod rewrite*

```
user@UbuntuWard:~$ sudo a2enmod rewrite
```

Restart Apache Service

Next, restart the Apache Service with the following command: *service apache2 restart*

```
user@UbuntuWard:~$ service apache2 restart
```

Create a test.php Web Page

To create a test.php web page, first enter this command: *sudo nano /var/www/html/test.php*

```
user@UbuntuWard:~$ sudo nano /var/www/html/test.php
```

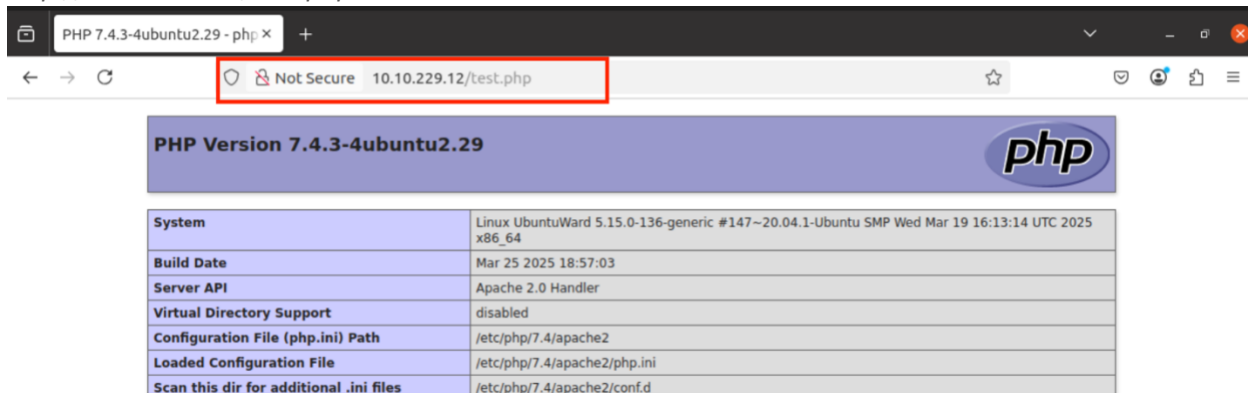
Next, insert the following test script then save and exit: `<?php phpinfo(); ?>`



Test the test.php Web Page

To test the web page, open the internet browser and enter the following into the address bar:

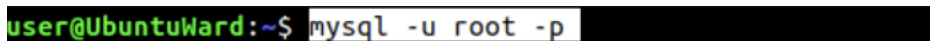
`http://10.10.229.12/test.php`



Database Configuration in MySQL

Log into MySQL

Enter the following command to log into MySQL: `mysql -u root -p`



Create MySQL WordPress Database (WordPressDB)

Create the database by entering the following SQL command:

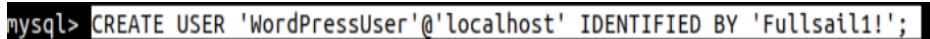
`CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;`



Create MySQL WordPress User (WordPressUser)

Next, create the MySQL WordPress user by entering the following command:

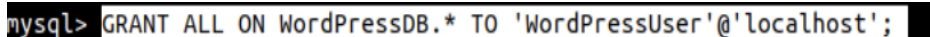
`CREATE USER 'WordPressUser'@'localhost' IDENTIFIED BY 'Fullsail1!';`



Grant Privileges to the WordPress Database (WordPressDB) to WordPress User (WordPressUser)

Next, to grant privileges to the WordPress database, enter this command:

`GRANT ALL ON WordPressDB.* TO 'WordPressUser'@'localhost';`



Flush Privileges

Now flush the privileges with the following command: *FLUSH PRIVILEGES;*

```
mysql> FLUSH PRIVILEGES;
```

Exit MySQL

Exit with this simple command: *exit*

```
mysql> exit
```

Install WordPress

Grant Permission to */var/www/html/* Directory to WordPress User

To grant permissions, enter the following command: *sudo chown \$USER:\$USER /var/www/html/**

```
user@UbuntuWard:~$ sudo chown $USER:$USER /var/www/html/*
```

Delete Files from */var/www/html/* Directory

Delete the files from the */var/www/html* directory with the following command:

*Sudo rm /var/www/html/**

```
user@UbuntuWard:~$ sudo rm /var/www/html/*
```

Verify */var/www/html/* Directory is Empty

Next, enter this command to verify the directory is empty: *sudo apt upgrade -y*

```
user@UbuntuWard:~$ sudo apt upgrade -y
```

Clone WordPress to */var/www/html/* Directory

Now, clone WordPress by entering the following command:

sudo git clone https://github.com/WordPress/WordPress /var/www/html/

```
user@UbuntuWard:~$ sudo git clone https://github.com/WordPress/WordPress /var/www/html/
```

Verify */var/www/html/* Directory Contains WordPress Files

To verify the installation, enter the following command: *sudo ls -la /var/www/html*

```
user@UbuntuWard:~$ sudo ls -la /var/www/html
```

Verify Permissions on */var/www/html/* Directory

Verify permissions with the following command: *sudo ls -ls /var/www/html*

```
user@UbuntuWard:~$ sudo ls -ls /var/www/html
```

Edit Ownership

Edit Ownership of and the contents of /var/www/html/ Directory

To edit ownership, you must enter two commands. First, enter this command:

*sudo chown -R www-data:www-data /var/www/html/**

```
user@UbuntuWard:~$ sudo chown -R www-data:www-data /var/www/html/*
```

Next, enter this command:

sudo chown www-data:www-data /var/www/html/

```
user@UbuntuWard:~$ sudo chown www-data:www-data /var/www/html/
```

Edit the apache2.conf File

To edit the configuration file, first enter the following command:

sudo nano /etc/apache2/apache2.conf --linenumbers

```
user@UbuntuWard:~$ sudo nano /etc/apache2/apache2.conf --linenumbers
```

Override All Default Apache Directives

Next, scroll down to lines 170-173 where you see "<Directory /var/www/>" and change "AllowOverride None" to "AllowOverride ALL" as shown below, then save and exit.

```
GNU nano 4.8 /etc/apache2/apache2.conf Modified
161 AllowOverride None
162 Require all denied
163 </Directory>
164
165 <Directory /usr/share>
166 AllowOverride None
167 Require all granted
168 </Directory>
169
170 <Directory /var/www/>
171 Options Indexes FollowSymLinks
172 AllowOverride ALL
173 Require all granted
174 </Directory>
175
176 #<Directory /srv/>
177 # Options Indexes FollowSymLinks
178 # AllowOverride None
179 # Require all granted
180 #</Directory>
181
182
183
184
```

Create a .htaccess File in the /var/www/html/.git/ Directory

To create the .htaccess file, first enter the following command:

sudo nano /var/www/html/.git/.htaccess

```
user@UbuntuWard:~$ sudo nano /var/www/html/.git/.htaccess
```

Then add the following two lines as shown below: "order deny, allow" and "deny from all"

```
user@UbuntuWard: ~
GNU nano 4.8 /var/www/html/.git/.htaccess Modified
order deny, allow
deny from all
```

Restart the Apache Service

Finally, restart the Apache service with the following command: `sudo systemctl restart apache2`

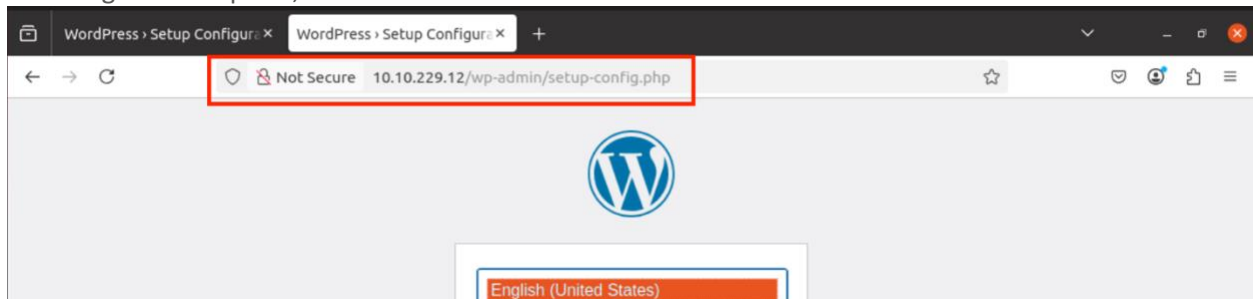
```
user@UbuntuWard:~$ sudo systemctl restart apache2
```

WordPress Configuration

Configure WordPress

WordPress Configuration Selections

To configure WordPress, first enter this IP address into the address bar: `10.10.229.12`



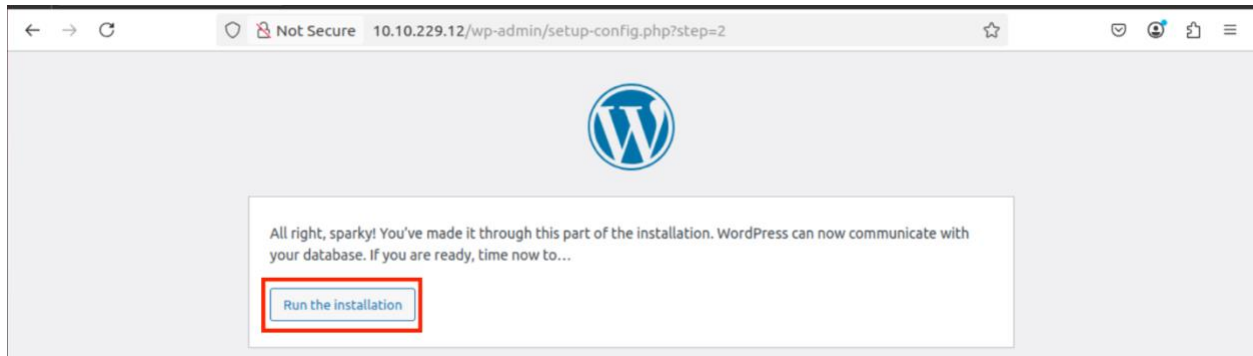
Next, proceed with the configuration by entering the following information as shown below:

- **Database Name:** WordPressDB
- **Username:** WordPressUser
- **Password:** Fullsail1!
- **Database Host:** localhost
- **Table Prefix:** wp_

A screenshot of the WordPress database configuration form. The form contains five input fields, each with a red box around its value: "Database Name" (WordPressDB), "Username" (WordPressUser), "Password" (Fullsail1!), "Database Host" (localhost), and "Table Prefix" (wp_). A "Submit" button is located at the bottom left of the form. The browser's address bar shows "10.10.229.12/wp-admin/setup-config.php?step=1".

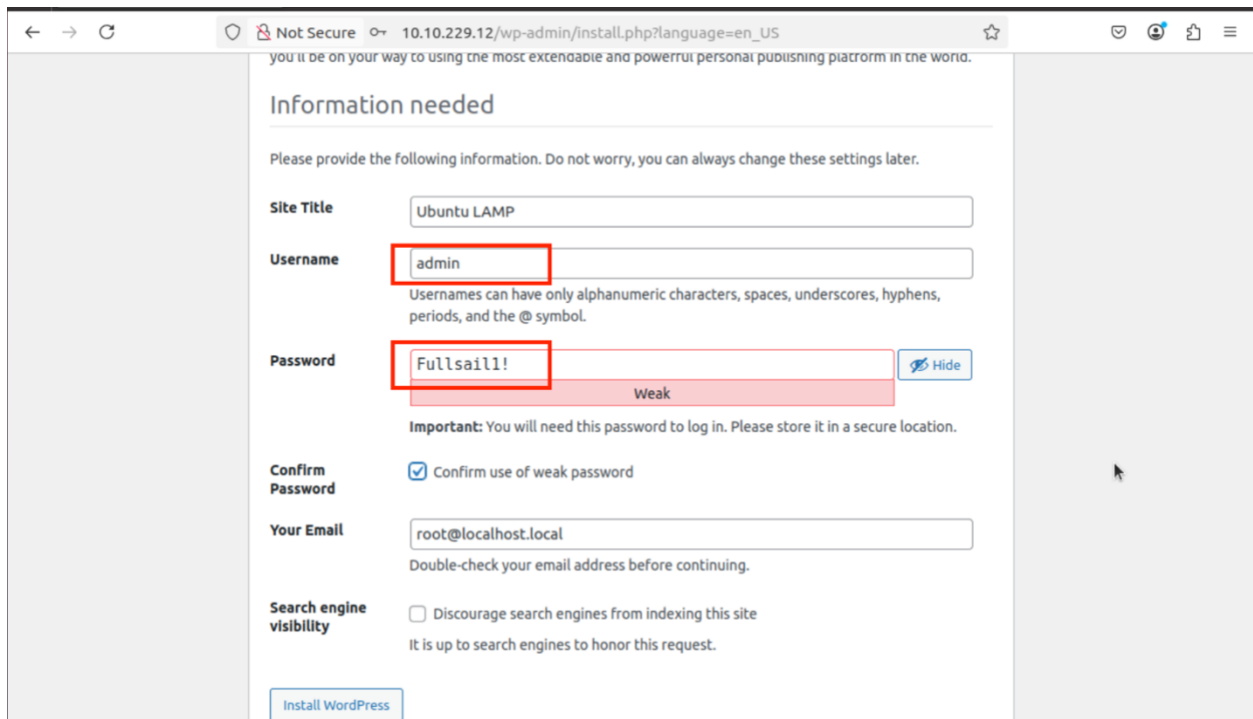
Run Installation

Now, run the installation by selecting “Run the installation”.



Create an Admin WordPress User

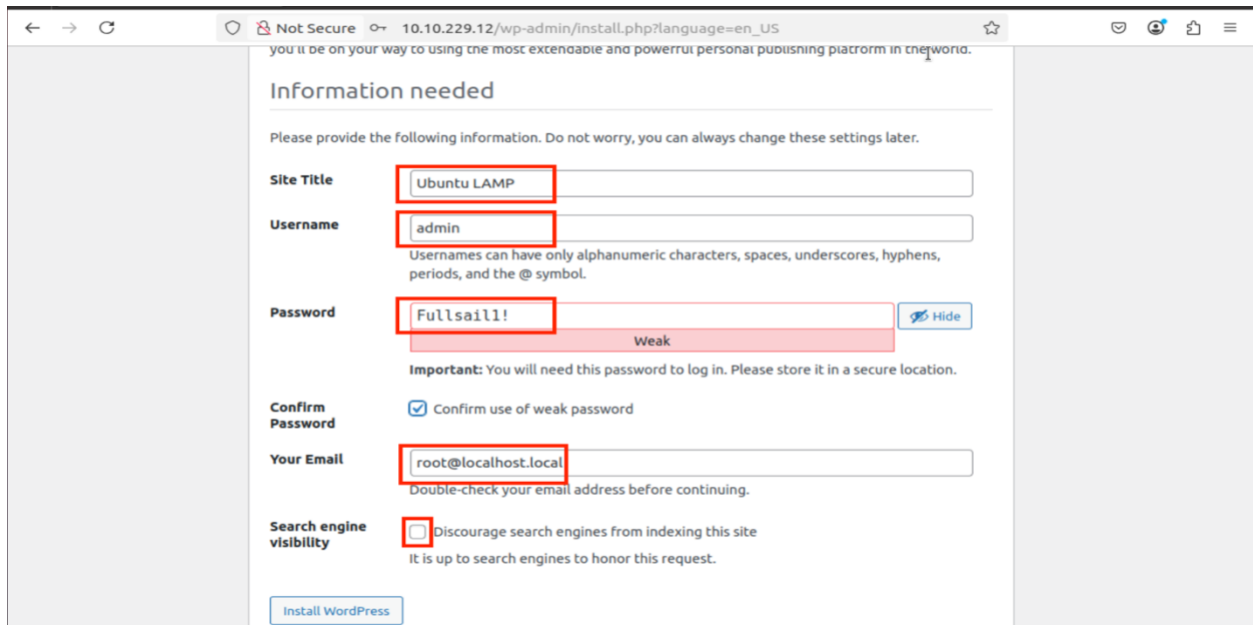
After running the installation, enter the admin user and password as shown below:



WordPress Site Selections

To properly configure the site selections, enter the rest of the information as shown below:

- **Site Title:** Ubuntu LAMP
- **Username:** admin
- **Password:** Fullsail1!
- **Your email:** root@localhost.local
- **Search Engine Visibility:** leave unchecked



Information needed

Please provide the following information. Do not worry, you can always change these settings later.

Site Title

Username
Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

Password [Hide](#)
Weak

Important: You will need this password to log in. Please store it in a secure location.

Confirm Password ☒ Confirm use of weak password

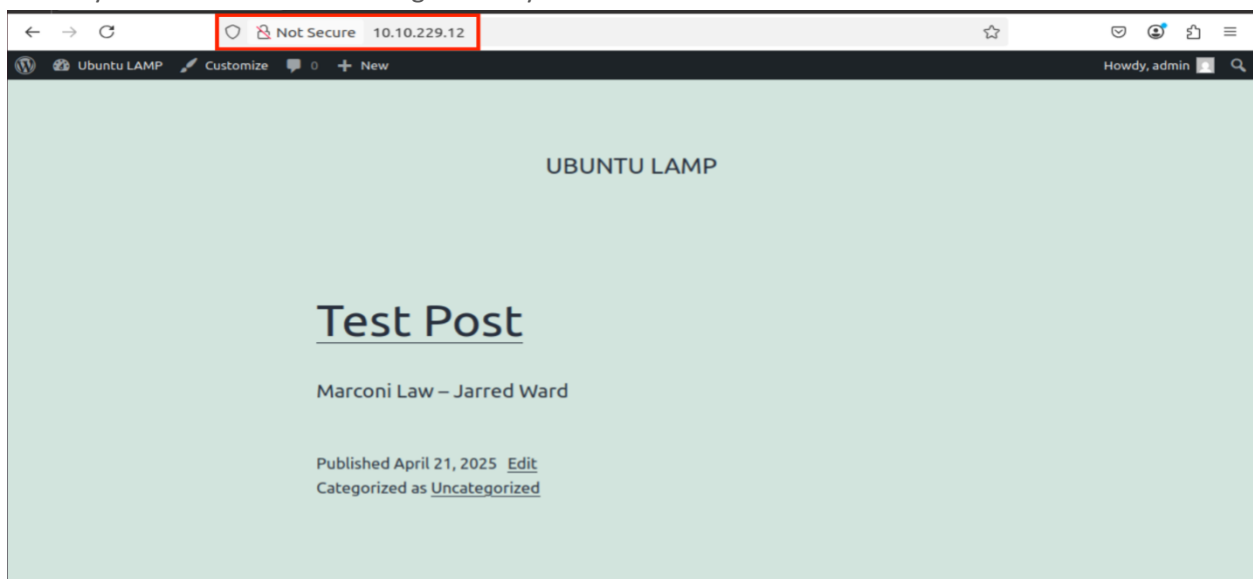
Your Email
Double-check your email address before continuing.

Search engine visibility ☐ Discourage search engines from indexing this site
It is up to search engines to honor this request.

[Install WordPress](#)

Test WordPress Website

After logging in with the admin account, create a test post and comment, then go to the following IP address to verify that WordPress is working correctly: 10.10.229.12



END OF MILESTONE 2

WordPress Security Settings and Configurations

File Permissions

The base Wordpress installation allows any user on the server to access all directories and read all files. These permissions need to be changed to improve the security of the website.

Vulnerability

Navigate to the WordPress installation and list its contents with the following command:

```
cd /var/www/html && ls -la
```

```
user@UbuntuWard:~$ cd /var/www/html && ls -la
total 252
drwxr-xr-x 6 www-data www-data 4096 Apr 23 21:08 .
drwxr-xr-x 3 root root 4096 Apr 17 18:18 ..
drwxr-xr-x 8 root root 4096 Apr 20 22:11 .git
-rw-r--r-- 1 www-data www-data 523 Apr 21 01:34 .htaccess
-rw-r--r-- 1 www-data www-data 405 Apr 20 21:24 index.php
-rw-r--r-- 1 www-data www-data 19903 Apr 20 21:24 license.txt
-rw-r--r-- 1 www-data www-data 7425 Apr 20 21:24 readme.html
-rw-r--r-- 1 www-data www-data 7387 Apr 20 21:24 wp-activate.php
drwxr-xr-x 9 www-data www-data 4096 Apr 20 21:24 wp-admin
-rw-r--r-- 1 www-data www-data 351 Apr 20 21:24 wp-blog-header.php
-rw-r--r-- 1 www-data www-data 2323 Apr 20 21:24 wp-comments-post.php
-rw-rw-rw- 1 www-data www-data 3625 Apr 21 01:18 wp-config.php
-rw-r--r-- 1 www-data www-data 3336 Apr 20 21:24 wp-config-sample.php
drwxr-xr-x 5 www-data www-data 4096 Apr 23 21:08 wp-content
-rw-r--r-- 1 www-data www-data 5617 Apr 20 21:24 wp-cron.php
drwxr-xr-x 30 www-data www-data 12288 Apr 20 21:24 wp-includes
-rw-r--r-- 1 www-data www-data 2502 Apr 20 21:24 wp-links-opml.php
-rw-r--r-- 1 www-data www-data 3937 Apr 20 21:24 wp-load.php
-rw-r--r-- 1 www-data www-data 51414 Apr 20 21:24 wp-login.php
-rw-r--r-- 1 www-data www-data 8727 Apr 20 21:24 wp-mail.php
-rw-r--r-- 1 www-data www-data 30081 Apr 20 21:24 wp-settings.php
-rw-r--r-- 1 www-data www-data 34516 Apr 20 21:24 wp-signup.php
-rw-r--r-- 1 www-data www-data 5102 Apr 20 21:24 wp-trackback.php
-rw-r--r-- 1 www-data www-data 3205 Apr 20 21:24 xmlrpc.php
user@UbuntuWard:/var/www/html$
```

Configuration

Enter three commands to configure the file permissions and list the new contents, as shown below:

```
sudo find /var/www/html/* -type d -exec chmod 770 {} \;
```

```
sudo find /var/www/html/* -type f -exec chmod 660 {} \;
```

```
ls -la
```

```
user@UbuntuWard:/var/www/html$ sudo find /var/www/html/* -type d -exec chmod 770 {} \;
user@UbuntuWard:/var/www/html$ sudo find /var/www/html/* -type f -exec chmod 660 {} \;
user@UbuntuWard:/var/www/html$ ls -la
total 252
drwxr-xr-x 6 www-data www-data 4096 Apr 24 08:44 .
drwxr-xr-x 3 root root 4096 Apr 17 18:18 ..
drwxr-xr-x 8 root root 4096 Apr 20 22:11 .git
-rw-r--r-- 1 www-data www-data 523 Apr 21 01:34 .htaccess
-rw-rw-rw- 1 www-data www-data 405 Apr 20 21:24 index.php
-rw-rw-rw- 1 www-data www-data 19903 Apr 20 21:24 license.txt
-rw-rw-rw- 1 www-data www-data 7425 Apr 20 21:24 readme.html
-rw-rw-rw- 1 www-data www-data 7387 Apr 20 21:24 wp-activate.php
drwxrwxr-x 9 www-data www-data 4096 Apr 20 21:24 wp-admin
-rw-rw-rw- 1 www-data www-data 351 Apr 20 21:24 wp-blog-header.php
-rw-rw-rw- 1 www-data www-data 2323 Apr 20 21:24 wp-comments-post.php
-rw-rw-rw- 1 www-data www-data 3625 Apr 21 01:18 wp-config.php
-rw-rw-rw- 1 www-data www-data 3336 Apr 20 21:24 wp-config-sample.php
drwxrwxr-x 5 www-data www-data 4096 Apr 23 21:08 wp-content
-rw-rw-rw- 1 www-data www-data 5617 Apr 20 21:24 wp-cron.php
drwxrwxr-x 30 www-data www-data 12288 Apr 20 21:24 wp-includes
-rw-rw-rw- 1 www-data www-data 2502 Apr 20 21:24 wp-links-opml.php
-rw-rw-rw- 1 www-data www-data 3937 Apr 20 21:24 wp-load.php
-rw-rw-rw- 1 www-data www-data 51414 Apr 20 21:24 wp-login.php
-rw-rw-rw- 1 www-data www-data 8727 Apr 20 21:24 wp-mail.php
-rw-rw-rw- 1 www-data www-data 30081 Apr 20 21:24 wp-settings.php
-rw-rw-rw- 1 www-data www-data 34516 Apr 20 21:24 wp-signup.php
-rw-rw-rw- 1 www-data www-data 5102 Apr 20 21:24 wp-trackback.php
-rw-rw-rw- 1 www-data www-data 3205 Apr 20 21:24 xmlrpc.php
user@UbuntuWard:/var/www/html$
```

Validation

Finally, test the configuration changes by attempting to navigate to the subdirectories as a user (not a root user). This is done by entering the following commands while still in the `/var/www/html` directory:

```
cd wp-admin
```

```
cd wp-content
```

```
cd wp-includes
```

```
user@UbuntuWard:/var/www/html$ cd wp-admin
bash: cd: wp-admin: Permission denied
user@UbuntuWard:/var/www/html$ cd wp-content
bash: cd: wp-content: Permission denied
user@UbuntuWard:/var/www/html$ cd wp-includes
bash: cd: wp-includes: Permission denied
user@UbuntuWard:/var/www/html$
```

Securing wp-config.php

The `wp-config.php` file needs to be moved out of the web-facing WordPress directory, to a more secure location that is not web-facing. We will move it to the parent directory.

Vulnerability

Navigate to the WordPress directory and list its contents to check for `wp-config.php` with this command:

```
cd /var/www/html && ls -la
```

```
user@UbuntuWard:~$ cd /var/www/html && ls -la
total 252
drwxr-xr-x 6 www-data www-data 4096 Apr 24 08:44 .
drwxr-xr-x 3 root root 4096 Apr 17 18:18 ..
drwxr-xr-x 8 root root 4096 Apr 20 22:11 .git
-rw-r--r-- 1 www-data www-data 523 Apr 21 01:34 .htaccess
-rw-rw---- 1 www-data www-data 405 Apr 20 21:24 index.php
-rw-rw---- 1 www-data www-data 19903 Apr 20 21:24 license.txt
-rw-rw---- 1 www-data www-data 7425 Apr 20 21:24 readme.html
-rw-rw---- 1 www-data www-data 7387 Apr 20 21:24 wp-activate.php
drwxrwx--- 9 www-data www-data 4096 Apr 20 21:24 wp-admin
-rw-rw---- 1 www-data www-data 351 Apr 20 21:24 wp-blog-header.php
-rw-rw---- 1 www-data www-data 2323 Apr 20 21:24 wp-comments-post.php
-rw-rw---- 1 www-data www-data 3625 Apr 21 01:18 wp-config.php
-rw-rw---- 1 www-data www-data 3336 Apr 20 21:24 wp-config-sample.php
drwxrwx--- 5 www-data www-data 4096 Apr 23 21:08 wp-content
-rw-rw---- 1 www-data www-data 5617 Apr 20 21:24 wp-cron.php
drwxrwx--- 30 www-data www-data 12288 Apr 20 21:24 wp-includes
-rw-rw---- 1 www-data www-data 2502 Apr 20 21:24 wp-links-opml.php
-rw-rw---- 1 www-data www-data 3937 Apr 20 21:24 wp-load.php
-rw-rw---- 1 www-data www-data 51414 Apr 20 21:24 wp-login.php
-rw-rw---- 1 www-data www-data 8727 Apr 20 21:24 wp-mail.php
-rw-rw---- 1 www-data www-data 30081 Apr 20 21:24 wp-settings.php
-rw-rw---- 1 www-data www-data 34516 Apr 20 21:24 wp-signup.php
-rw-rw---- 1 www-data www-data 5102 Apr 20 21:24 wp-trackback.php
-rw-rw---- 1 www-data www-data 3205 Apr 20 21:24 xmlrpc.php
user@UbuntuWard:/var/www/html$
```

Configuration

Move the `wp-config.php` file to the parent direct with the following command:

```
sudo mv wp-config.php /var/www/
```

```
user@UbuntuWard:/var/www/html$ sudo mv wp-config.php /var/www/
```

Validation

To verify the change in configuration, navigate to the parent directory and list its contents to see that the wp-config.php file has been moved with this command: `cd /var/www && ls -la`

```
user@UbuntuWard:/var/www/html$ cd /var/www && ls -la
total 16
drwxr-xr-x  3 root    root    4096 Apr 24 11:25 .
drwxr-xr-x 15 root    root    4096 Apr 17 18:18 ..
drwxr-xr-x  6 www-data www-data 4096 Apr 24 11:25 html
-rw-rw----  1 www-data www-data 3625 Apr 21 01:18 wp-config.php
user@UbuntuWard:/var/www$
```

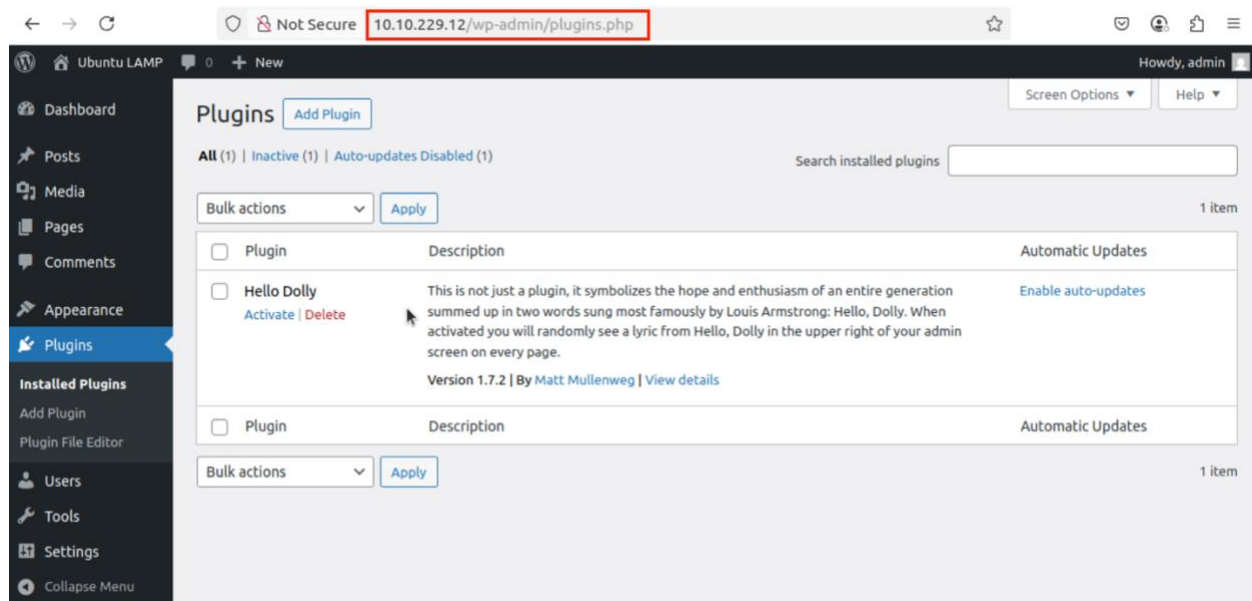
Firewall (Shield)

The base WordPress installation has no firewall. We will add an application firewall to improve security.

Vulnerability

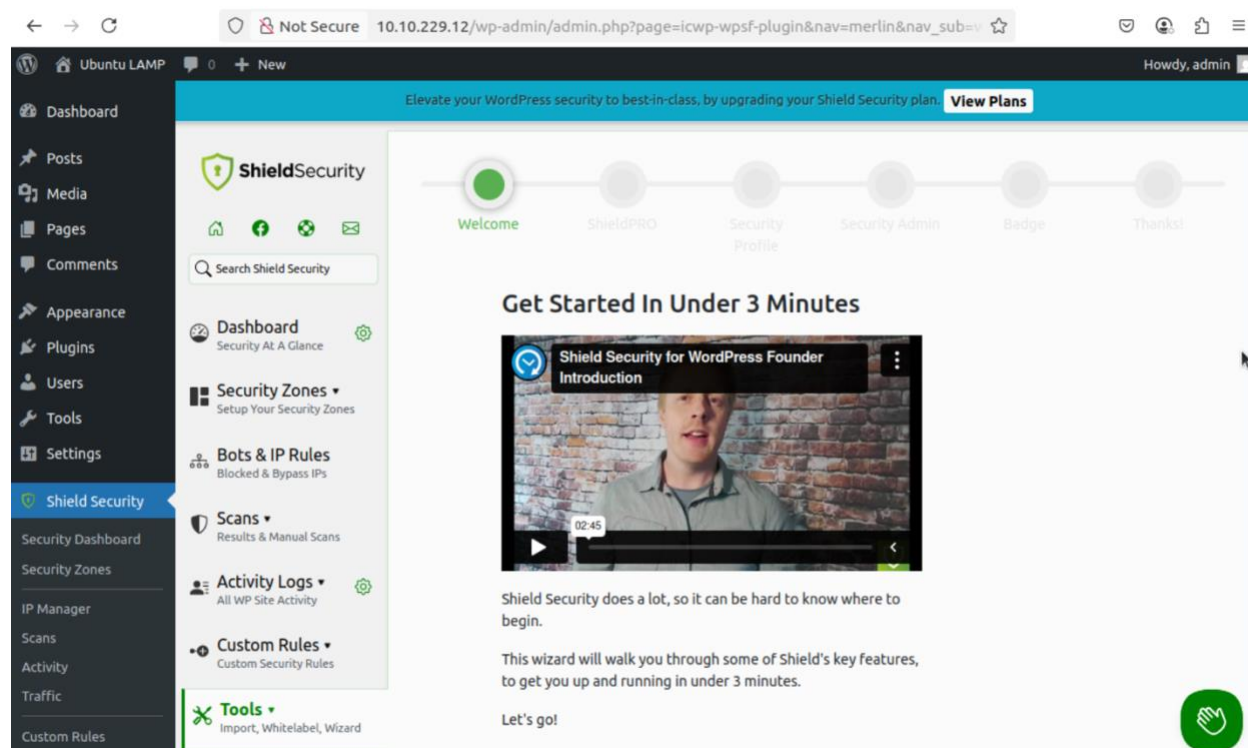
Navigate to the WordPress plugins page at this address to see that there is no firewall installed:

`10.10.229.12/wp-admin/plugins.php`



Configuration

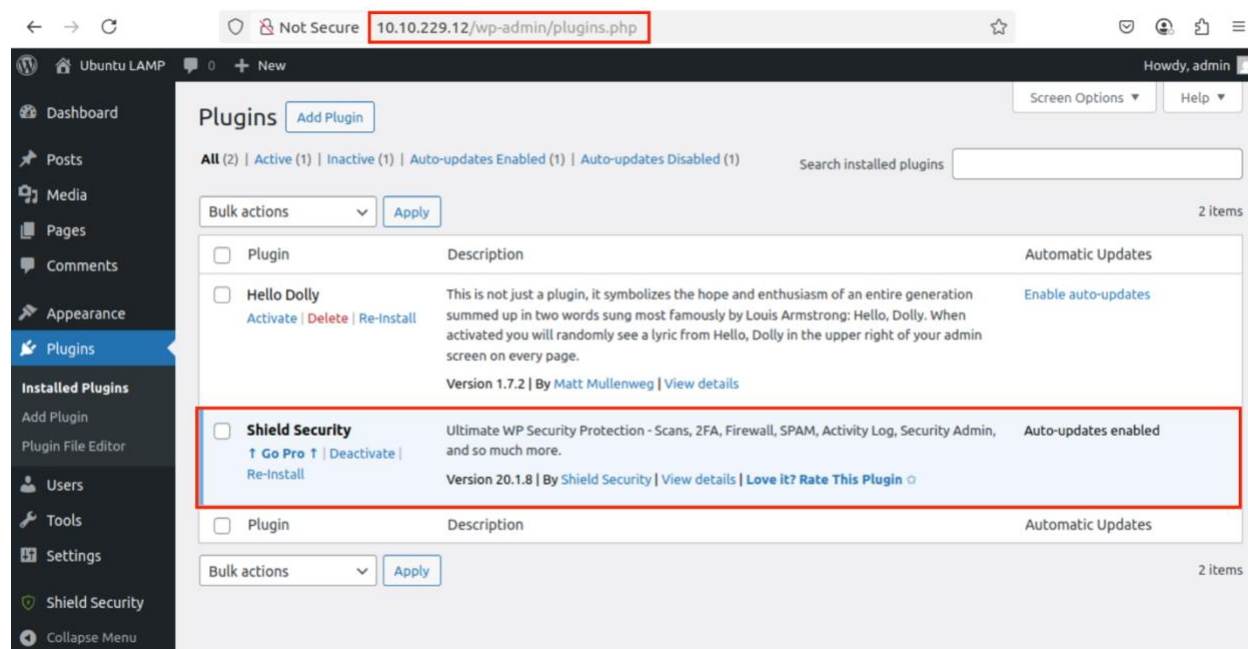
Install, activate, and configure the Shield Security firewall as shown below:



Validation

Return to the installed plugins page to verify that the configuration is complete:

10.10.229.12/wp-admin/plugins.php



Conclusion

To secure my WordPress blog site, I made a series of key changes to improve its overall security. First, I adjusted the default file permissions in the WordPress directory (*/var/www/html*). This included modifying the permissions for the *wp-admin*, *wp-content*, and *wp-includes* directories, along with all files within the WordPress installation. By restricting read, write, and execute permissions, I limited the potential for unauthorized access or modification. A second step involved securing the *wp-config.php* file, which was stored in the web-accessible WordPress directory. To protect it from potential attacks, I moved it to a more secure location, specifically to the parent directory */var/www/*. Lastly, I installed a Layer 7 firewall plugin to filter out malicious traffic at the application level. After activating and configuring the plugin, I verified its installation through the WordPress dashboard, ensuring the plugin was active and functioning.

I applied the concept of defense-in-depth throughout my WordPress site's security setup. Defense-in-depth is a strategy that layers multiple security measures to protect a system. This is crucial because it reduces the likelihood of a single vulnerability compromising an entire system. By implementing various security measures at different levels, attackers must penetrate multiple security barriers. This increases the total effort required to breach the system. This layered approach provides redundancy, ensuring that if one defense fails, others remain in place to mitigate potential damage. In this case, I used several layers. The first was adjusting file permissions to control access to critical files. The second layer was moving the *wp-config.php* file out of the web root directory, which reduces the risk of direct exposure. The third layer was the firewall plugin, which helps protect against web-based attacks like SQL injection or cross-site scripting. All of these layers work together to provide a more comprehensive defense.

The strategies I implemented to secure my WordPress site can be applied to the installation, configuration, and securing of other systems as well. For instance, when setting up a new server, securing the file system should always be a priority by adjusting permissions based on the principle of least privilege. In the case of a web application, it's important to ensure that critical configuration files are not stored in publicly accessible directories, like how I moved the *wp-config.php* file. Additionally, installing firewalls or other intrusion detection/prevention tools should be a standard practice when configuring any new system. These tools can provide an extra layer of defense against common attacks like DDoS or SQL injection. Beyond technical configurations, it's also vital to keep all systems regularly updated, since the overwhelming majority of attacks exploit known vulnerabilities in outdated software. By applying these security strategies universally, I can create a more secure environment and significantly reduce the risk of unauthorized access or exploitation.

Appendix A

NginX Config File

```
root@DockerWard:~  
root@NginxWard:~  
GNU nano 2.9.8 /etc/nginx/nginx.conf  
  
# Load modular configuration files from the /etc/nginx/conf.d directory.  
# See http://nginx.org/en/docs/nginx_core_module.html#include  
# for more information.  
include /etc/nginx/conf.d/*.conf;  
  
server {  
    listen      80 default_server;  
    listen      [::]:80 default_server;  
    server_name _;  
    root        /usr/share/nginx/html;  
  
    # Load configuration files for the default server block.  
    include /etc/nginx/default.d/*.conf;  
  
    location / {  
    }  
  
    location /blog {  
        proxy_pass          http://10.10.229.11:3001;  
        proxy_set_header    Host      $http_host;  
        proxy_set_header    X-Real-IP $remote_addr;  
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_read_timeout   900;  
    }  
  
    error_page 404 /404.html;  
        location = /40x.html {  
        }  
  
    error_page 500 502 503 504 /50x.html;  
        location = /50x.html {  
        }  
    }  
}
```

Appendix B

NginX Access Log File

```
[root@NginxWard nginx]# tail access.log  
10.10.229.12 - - [12/Apr/2025:00:45:40 -0400] "GET /blog HTTP/1.1" 404 3332 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0" "-"  
10.10.229.12 - - [12/Apr/2025:00:45:41 -0400] "GET /nginx-logo.png HTTP/1.1" 200 368 "http://10.10.229.10/blog" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0" "-"  
10.10.229.12 - - [12/Apr/2025:00:45:41 -0400] "GET /poweredby.png HTTP/1.1" 200 1800 "http://10.10.229.10/blog" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0" "-"  
10.10.229.12 - - [12/Apr/2025:00:45:41 -0400] "GET /favicon.ico HTTP/1.1" 404 3332 "http://10.10.229.10/blog" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0" "-"
```

NginX Error Log File

```
[root@NginxWard nginx]# tail error.log  
2025/04/12 00:34:29 [emerg] 8074#0: directive "location" has no opening "[" in /etc/nginx/nginx.conf:51  
2025/04/12 00:45:40 [error] 8075#0: *1 open() "/usr/share/nginx/html/blog" failed (2: No such file or directory), client: 10.10.229.12, server: _, request: "GET /blog HTTP/1.1", host: "10.10.229.10"  
2025/04/12 00:45:41 [error] 8075#0: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 10.10.229.12, server: _, request: "GET /favicon.ico HTTP/1.1", host: "10.10.229.10", referer: "http://10.10.229.10/blog"  
[root@NginxWard nginx]#
```